# Closure Objects and set

Lecture 2 Private state and explicit binding updates

Gregory S. DeLozier, PhD

# What ordinary assignment does

```
x = 10;
function changeLocally() {
    x = x + 1;
    return x;
};
print(changeLocally());
print(changeLocally());
print(x);
```

# An explicit update statement

```
x = 10;
function changeOuter() {
    set x = x + 1;
    return x;
};
print(changeOuter());
print(changeOuter());
print(x);
```

# The rules for set

set x = expression;

Search the current environment, then its parents. Update the first existing binding of x. Raise an error if x does not exist.

Ordinary x = expression writes locally.

# Nearest binding wins

```
x = 100;
function outer() {
    x = 10;
    function inner() { set x = x + 1 };
    inner();
    return x;
};
print(outer());
print(x);
```

# set respects shadowing

```
x = 100;
function outer() {
    x = 10;
    function inner(x) {
        set x = x + 1;
        return x;
    };
    print(inner(3));
    return x;
};
print(outer());
print(x);
```

Prints 4, then 10, then 100

# A counter with a captured binding

```
function makeCounter() {
    count = 0;
    return function() {
        set count = count + 1;
        return count;
    };
};
next = makeCounter();
print(next());
print(next());
```

# Two methods, one private environment

```
function makeCounter() {
    count = 0;
    return {
        "increment": function() {
            set count = count + 1;
            return count;
        },
        "value": function() { return count }
    };
}
```

# Independent instances

```
a = makeCounter();
b = makeCounter();
print(a.increment());
print(a.increment());
print(a.value());
print(b.value());
```

One factory call creates one private count binding.

# The object correspondence

Factory call: construction

Retained bindings: instance state

Returned functions: methods

Returned record: public interface

# An extracted method keeps its state

```
a = makeCounter();
increment = a.increment;
readValue = a.value;
print(increment());
print(readValue());
```

The closure carries the connection to its instance.

# Returning capabilities

```
readValue = a.value;
// Pass readValue to another function.
// It can observe this counter through that function.
```

An interface can expose selected operations. Returning the state itself exposes more access.

# Binding update and object mutation

```
// Update an existing captured binding
set count = count + 1;

// Mutate data reached through a binding
state.count = state.count + 1;

// Assign in the current environment
count = count + 1;
```

# Parsing set

```
set_statement ::= "set" identifier "=" expression

{"tag": "set",
 "name": name,
 "expression": expression}
```

A statement with an identifier target

# Evaluating set

```
destination = environment
while name not in destination:
    if PARENT_KEY not in destination:
        raise NameError(...)
    destination = destination[PARENT_KEY]
value, status = evaluate(expression, environment)
if status is not None:
    return value, status
destination[name] = value
```

# Practice: reset and step

Extend makeCounter(start, step).

next() adds step and returns the new value. value() reads the current value. reset() restores start.

What should two independent instances print?

# Design questions

Should an update create a missing variable?

Which binding should shadowing select?

Which references should a factory expose?

How long can a retained environment remain reachable?